WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Lucas Andrew

Email: mailto:andrewl2@mail.uc.edu

Short-bio: Lucas Andrew has 7 years of industry experience. He has experience with web development and low-level programming. Currently, Lucas works in the Cyber domain and regularly programs in C and Python.

Individual Project 1 - Front-end Web Development with a Professoinal Profile Website and API Integration on github.io cloud service

Overview and Requirements

This project had me create a website containing my resume and published it to the github.io cloud service where anyone who has the link can view this website. To create this website, I was requried to use the skills I learned in this course so far. In this assignment, I learned how to integrate an open-source CSS

framework so the website can look nice to the end-user. I also learned how to use JavaScript cookies to track when a user last visited my website and display them a message with this information. All other exercises, including integrating a JavaScript framework of choice, I have learned previously.

My Professional Profile link: https://lucasandrew2.github.io.

Individual Project 1 link: https://github.com/lucasandrew2/lucasandrew2.github.io.

General Requirements

The General Requirements given in this project were addressed in the title section of my website (in the block with a green background) and the "Resume" section of the website (in the block with a white background). Please note that a link to a new HTML page to introduce the "Web Application Programming and Hacking" course is under the "Relevant Courses" section.

Non-Technical Requirements

The first Non-Technical Requirement in this project was addressed by integrating the Bootstrap "Simply Me" CSS template in my code. I did this by including a script to download this theme and then linking the downloaded stylesheet to my website. This allowed the theme to be applied throughout my website. Additionally, I used themes provided in the W3Schools example. I also used some of the custom CSS styles included in the example. These were the CSS classes `bg-1`, `bg-2`, `bg-3`, and `container-fluid`. I added one more CSS class of my own which helped make the UI look better in the final page called `no-bottom-margin` that I applied to some elements.

The final Non-Technical Requirement in this project was addressed by including a page tracker at the bottom of my website. For this, I used the Flag Counter given as one of the suggestions for us to use. This website gave me pastable HTML source to include the tracker. I verified that it was correct that it displayed the USA flag with a value of 1 during development since I was the only one who navigated to the website at that time.

Technical Requirements

jQuery + AngularJS The first Technical Requirement was to use jQuery and an open-source JavaScript library to implement a digital clock, an analog clock, showing and hiding my email, and another functionality of my choice.

Both clocks can be found in the title section of my website (in the block with a green background). Functionality to show and hide my email can be found under the "My Email" section in the "Resume" section (in the block with a white background). These were implemented using jQuery.

Finally, I included functionality to display a random motivational quote about programming after clicking a button. This can be found in the "Motivational

Quote" section (in the block with a dark blue background). For this feature, there are a total of 5 motivational quotes that can be displayed. This was implemented using AngularJS. I chose AngularJS since I have previous experience using this framework at work. I implemented this functionality with AngularJS's data binding and event handling. Below is the AngularJS code I used to implement the feature.

```
<div ng-app="quote-app" ng-controller="quote-controller">
    <h3>Motivational Quote</h3>
    <p>{{ quote }}</p>
    <button ng-click="getQuote()">
        New Quote
    </button>
    <script>
        var app = angular.module('quote-app', []);
        app.controller('quote-controller', function($scope) {
            $scope.quotes = [
                "Success is the sum of small efforts repeated daily.",
                "First solve the problem, then write the code.",
                "Programming is not about what you know, it is about what you can figure out
                "while (! ( succeed = try() ) );",
                "Simplicity is prerequisite for reliability.",
            ];
            $scope.quote = $scope.quotes[0];
            $scope.getQuote = function() {
                var index = Math.floor(Math.random() * $scope.quotes.length);
                $scope.quote = $scope.quotes[index];
            };
        });
    </script>
</div>
```

In AngularJS, an app must first be created and then a controller must be added to it. The controller is responsible for controlling the data flow and managing user interactions. It acts as a mediator between the HTML and the data. To access HTML elements, a special object called `$scope` must be used. Here, I use it to set the value of `quote` and to create a `getQuote` function that is called upon every button click.

Note: since there are 5 quotes and each quote is randomly obtained, there is a 20% possibility that the same quote is picked from a button click.

API Integration The second Technical Requirement was to integrate two public APIs was addressed in the "API Integrations" section of my website (in the block with a dark blue background). This section contains a random joke from the `Any` category of the jokeAPI every minute and a random picture of a German Shepherd (my favorite dog breed) every time the page is navigated to

from the dog-api.

I used the JavaScript function `setInterval(getJoke, 60000)`. To get a new joke every minute. Note that `setInterval` takes a time in milliseconds.

JavaScript Cookies The final Technical Requirement was to add a JavaScript cookie that displays the message "Welcome to my homepage for the first time" or "Welcome back! Your last visit was (time-of-last-visit)". I implemented this by using the example code provided from Lecture 6. However, to set the "Welcome back" message, I used a JavaScript `Date` object to get the current time in a nice-to-read format and appended it to the base message.

This functionality can be tested by navigating to my website, where an alert for this message will be displayed to the user upon load. Note that the time of last visit is appropriately updated upon subsequent visits.